

Q1: Lists vs Arrays

Conceptual difference:

Python list = array of pointers

- each element in the list points to an object in memory.

Low-level array = contiguous block of memory

- all element stored directly in memory.

Advantage in Lists can allow different datatypes.

Disadvantage in Lists is more memory usage.

Advantage in Arrays is memory efficient.

Disadvantage in Arrays can only datatype.

Q2: Strings & Immutability

Python strings are immutable; String is created only once; it can't be changed. Because strings are often used as key in dictionaries or elements in set, and memory optimization; python can refer to same memory location by different variable if its same value.

If you want to change string, use new string and slice of another string.

Time Complexity of string slicing is $O(\text{size of slice})$

Time Complexity of concatenation $O(\text{sum of length of two string}) \rightarrow O(n+m)$

Q3: Dictionaries

Dictionaries are fast because using hash table [Direct Access].

Hash collision happens when two different key produce the same index, and handle this in python by (Open Addressing); Python checks the next available index, or continue checking until gain an empty index.

Average time in lookup times is $O(1)$ and worst-case is $O(n)$, $n \rightarrow$ number of element.

Q4: Sets vs Lists

We prefer set over a list when we need unique elements only, and order doesn't important. Ex(union, intersection, and difference)

Examples: checking if a username is already taken, and Remove duplicates

Q5: Stack, Queue & Deque

Collections.deque prefer for queues because time complexity when remove at beginning in collections.deque is $O(1)$ but in list is $O(n)$.

Insert or Remove at beginning is expensive operation on lists because time complexity is $O(n)$.

Real-world use cases:

Stack (FILO) → Undo, and Back Button.

Queue (FIFO) → Waiting Line, and Scheduling.